Notebook themes

Theme bundles can style notebook HTML output and paged output. For a single document, select a theme with `--theme` or `#calepin.setup(theme: ...)`:

```
calepin compile paper.typ --theme calepin
```

`--theme` is optional. Builds use the builtin `calepin` bundle by default. The builtin `academic` bundle customizes website pages and falls back to `calepin` for single-document HTML and paged output.

HTML themes

HTML themes use MiniJinja templates plus optional CSS and JavaScript files. For notebooks compiled as single documents, the relevant template is `document.html`.

Local HTML themes are selected by pointing `--theme`, a website `theme` setting, or `#calepin.setup(theme: ...)` at a theme bundle:

```
themes/my-theme/
  document.html
  site.html
  partials/
  styles/
  scripts/
```

`document.html` and `site.html` are MiniJinja templates. `partials/` files can be included with `{% include "partials/name.html" %}`. CSS files in `styles/` and JavaScript files in `scripts/` are loaded in filename order and exposed as `styles` and `scripts` arrays.

Templates can access:

- `doc.head`
- `doc.body_open`
- `doc.body`
- `doc.body_close`
- `doc.title`
- `site.sidebar`
- `site.sidebar_sections`
- `site.navbar_left`
- `site.navbar_center`
- `site.navbar_right`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.favicon`
- `site.current_url`
- `site.page_title`
- `styles`

- `scripts`
- `syntax_css`
- `theme`
- `target`

Navigation entries expose `href`, `label`, `label_html`, and `active`.

Here is a minimal `document.html` for single-file HTML output:

```
{{ doc.head }}
  <title>{{ doc.title }}</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.
min.css">
  {% for style in styles %}
  <style>
{{ style.css }}
  </style>
  {% endfor %}
{{ doc.body_open }}
  <header class="document-header">
    <a href="index.html">Home</a>
    <button type="button" data-calepin-theme-toggle>Theme</button>
  </header>
  <main class="container">
    {{ doc.body }}
  </main>
  {% for script in scripts %}
  <script>
{{ script.content }}
  </script>
  {% endfor %}
{{ doc.body_close }}
```

`doc.head`, `doc.body_open`, and `doc.body_close` come from Typst's generated HTML. Keep them in the template unless you are deliberately replacing the whole document shell.

Shared CSS and JavaScript are exposed as normal files through `styles` and `scripts`. Built-in themes and newly ejected themes use files such as `styles/00-theme.css`, `styles/01-code.css`, `styles/02-widgets.css`, `scripts/00-theme-toggle.js`, `scripts/01-language-picker.js`, and `scripts/02-copy-code.js`.

`styles/00-theme.css` is the shared visual base used by `calepin` and `academic`. It defines common typography, heading sizes, accent variables, Pico primary colors, code/output variables, figure defaults, and global document defaults. Theme-specific CSS should generally be limited to the HTML shell and layout differences that cannot be shared.

`styles/02-widgets.css` pairs with the shared JavaScript files:

- `scripts/00-theme-toggle.js` enhances buttons marked with `data-calepin-theme-toggle`
- `scripts/01-language-picker.js` enhances selects marked with `data-calepin-language-picker`
- The website view switcher expects a select with `id="calepin-website-view-mode"`

PDF themes

PDF, SVG, and PNG notebook outputs use a paged MiniJinja template named `paged.typ.jinja`:

```
themes/my-theme/
  paged.typ.jinja
```

Typst already has a complete styling system based on `#set` and `#show` rules. Calepin's paged theme layer adds one optional step before Typst runs: `paged.typ.jinja` is rendered with MiniJinja and must produce Typst source. This keeps the customization model similar to HTML themes, exposes Calepin-specific values, and still lets Typst handle the actual PDF/SVG/PNG styling.

The bundled `calepin` theme's `paged.typ.jinja` is enabled by default for `calepin compile` and website PDF/SVG/PNG builds. To customize it, eject the default bundle:

```
calepin new theme
```

Then edit `themes/calepin/paged.typ.jinja` and select that local theme:

```
calepin compile paper.typ --theme themes/calepin
```

In a website, use the same local theme directory in `calepin.toml`:

```
theme = "themes/calepin"
```

Internally, Calepin uses the paged template while preparing the Typst file that Typst will compile:

1. Calepin preprocesses the notebook and writes a staged Typst source file under `.calepin/`.
2. Calepin renders `paged.typ.jinja` with MiniJinja.
3. `document.body` expands to a Typst `#include` for the staged notebook source.
4. Calepin writes a wrapper file that contains the rendered theme and any notebook execution rules.
5. Typst compiles that wrapper to PDF, SVG, or PNG.

For example, this template:

```
#set page(numbering: "1")

{{ document.body }}

[#align(center)[Generated by Calepin]]
```

becomes Typst source like this:

```
#set page(numbering: "1")

#include "/.calepin/paper/source.typ"

[#align(center)[Generated by Calepin]]
```

The exact `.calepin/.../source.typ` path is generated by Calepin. You normally do not write that path yourself; use `{{ document.body }}`.

The template can be mostly plain Typst when no Calepin variables are needed. Use `document.body` where the notebook source should appear:

```
#set page(
  paper: "us-letter",
  margin: (x: 1in, y: 0.85in),
  numbering: "1",
)

#set text(font: "Libertinus Serif", size: 10.5pt)
#set heading(numbering: "1.1")

#show heading.where(level: 1): it => {
```

```
  pagebreak(weak: true)
  text(size: 18pt, weight: "semibold", it)
}

#show raw.where(block: true): it => {
  if it.theme != auto {
    it
  } else {
    block(
      width: 100%,
      fill: rgb("#f7f7f5"),
      stroke: 0.5pt + rgb("#d8d8d2"),
      inset: (x: 0.65em, y: 0.45em),
      radius: 2pt,
    )[#it]
  }
}

{{ document.body }}
```

Because the rendered output is Typst, it can also import project files such as `#import "/styles/report.typ": *`. Root-relative imports start from the website or document root.

`paged.typ.jinja` receives:

- `theme`: the local theme directory name
- `target`: `paged`
- `document.path`: the root-relative `.typ` input path
- `document.dir`: the root-relative input directory
- `document.stem`: the input filename without `.typ`
- `document.body`: the staged notebook source as a Typst `#include`
- `document.meta`: metadata from `#metadata(...) <website-metadata>`
- `params`: document parameters after CLI overrides

Use those values to insert generated front matter, appendices, disclaimers, or labels from Calepin metadata:

```
{% if document.meta.title %}
#set page(footer: align(right)[{{ document.meta.title }}])
{% endif %}

{{ document.body }}

{% if document.meta.appendix %}
pagebreak(weak: true)
heading("Appendix")
{{ document.meta.appendix }}
{% endif %}
```

If `paged.typ.jinja` does not reference `document.body`, Calepin treats it like a prelude and includes the notebook source after the rendered template.

For output-specific branches, use Typst's runtime input instead of MiniJinja:

```
#let is-html = sys.inputs.at("calepin-target", default: "paged") == "html"
```

Set `theme = false` or use an empty `paged.typ.jinja` to disable paged styling.